

# O que é Selenium?

**Selenium** é uma **ferramenta de automação de testes para aplicações web**. Ele permite que você **simule ações de um usuário real no navegador**, como clicar em botões, preencher formulários, navegar entre páginas, entre outros — tudo isso de forma **automatizada**.

O Selenium é amplamente utilizado por **desenvolvedores e times de QA (Quality Assurance)** para:

1. **Testes automatizados de aplicações web**
  - Testes funcionais (clicar, preencher, validar dados)
  - Testes de login, navegação, formulários, filtros, etc.
2. **Testes de regressão**
  - Verifica se novas versões do software não introduzem erros antigos
3. **Testes em múltiplos navegadores**
  - Chrome, Firefox, Edge, Safari (cross-browser testing)
4. **Integração com ferramentas de CI/CD**
  - Jenkins, GitHub Actions, GitLab CI (para rodar testes automaticamente a cada commit)

## ✓ 1. Pré-requisitos para criar testes com selenium

Certifique-se de ter o seguinte instalado:

- **Java JDK** (Java 8 ou superior)
- **Eclipse IDE for Java Developers**
- **Maven** (opcional, mas recomendado)

---

## ✓ 2. Criar um Projeto no Eclipse com Maven

1. Abra o Eclipse.
2. Vá em **File > New > Maven Project**.
3. Selecione um arquétipo simples (maven-archetype-quickstart).
4. Defina o **Group Id** e **Artifact Id** (ex: br.com.exemplo e selenium-test).
5. Finalize e aguarde o projeto ser criado.

---

## ✓ 3. Adicionar Dependência do Selenium no pom.xml

Abra o pom.xml e adicione:

```
<dependencies>
  <!-- Selenium WebDriver -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
```

```
        <artifactId>selenium-java</artifactId>
        <version>4.20.0</version>
    </dependency>
</dependencies>
```

Depois clique com o botão direito no projeto > **Maven > Update Project.**

---

## ✓ 4. Exemplo de Teste Simples com Selenium

Aqui está um teste que abre o site do Google e verifica o título da página:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class GoogleTest {

    WebDriver driver;

    @Before
    public void setUp() {
        // Caminho do driver do Chrome
        System.setProperty("webdriver.chrome.driver",
"caminho/para/chromedriver.exe");
        driver = new ChromeDriver();
    }

    @Test
    public void testGoogleTitle() {
        driver.get("https://www.google.com");
        String title = driver.getTitle();
        assertEquals("Google", title);
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit(); // Fecha o navegador
        }
    }
}
```

---

## ✓ 5. Baixar o ChromeDriver

1. Acesse: <https://sites.google.com/chromium.org/driver/>
  2. Baixe a versão do **ChromeDriver** compatível com seu navegador.
  3. Extraia o arquivo e salve o caminho para usar em `System.setProperty`.
-

Se você está usando o **navegador Microsoft Edge** com Selenium no Java, precisará do **Edge WebDriver**, também conhecido como **msedgedriver**, que é o equivalente do `chromedriver`,

---

## ✓ 1. Verifique a Versão do Microsoft Edge

Abra o Edge e vá em:

Menu (três pontos) > Ajuda e feedback > Sobre o Microsoft Edge

Anote a versão (ex: 123.0.2420.65).

---

## ✓ 2. Baixe o Edge WebDriver (msedgedriver)

Acesse:

🔗 <https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/>

- Clique na versão que **corresponde à do seu navegador Edge**.
  - Faça o download do `msedgedriver.exe`.
  - Extraia e **salve em uma pasta do seu sistema** (ex: `C:\drivers\`).
- 

## ✓ 4. Exemplo Completo com Edge

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;

public class EdgeTest {

    WebDriver driver;

    @Before
    public void setUp() {
        System.setProperty("webdriver.edge.driver",
"C:\\\\drivers\\\\msedgedriver.exe");
        driver = new EdgeDriver();
    }

    @Test
    public void testEdgeGoogleTitle() {
        driver.get("https://www.google.com");
```

```
        String title = driver.getTitle();
        assertEquals("Google", title);
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

---

## ✓ Automatize com WebDriverManager (funciona com Edge também!)

Adicione no `pom.xml`:

```
<dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.7.0</version>
</dependency>
```

E no código:

```
import io.github.bonigarcia.wdm.WebDriverManager;

@Before
public void setUp() {
    WebDriverManager.edgedriver().setup();
    driver = new EdgeDriver();
}
```

---

O **WebDriver** é um **componente central do Selenium**. Ele atua como um **"controlador" que automatiza o navegador**, permitindo que você escreva testes (ou scripts) que simulam ações reais de um usuário, como:

- Acessar uma página
  - Clicar em botões
  - Preencher formulários
  - Ler textos da tela
  - Validar resultados
  - Navegar entre abas, rolar a página, etc.
- 

## ✓ De forma simples:

O **WebDriver** é a ponte entre seu código Java e o navegador real.

---

## 🔧 Como ele funciona?

1. **Você escreve o teste em Java**, usando comandos como `driver.get()`, `driver.findElement()` etc.
  2. O Selenium WebDriver **envia comandos para o navegador** usando o driver correspondente:
    - o `ChromeDriver` para o Chrome
    - o `EdgeDriver` para o Edge
    - o `FirefoxDriver` para o Firefox
  3. O WebDriver **executa as ações reais no navegador**, como um humano faria.
- 

## ✈️ Exemplo prático

```
WebDriver driver = new EdgeDriver(); // Inicia o navegador Edge
driver.get("https://www.google.com"); // Abre o site do Google
driver.findElement(By.name("q")).sendKeys("Selenium"); // Digita na
busca
driver.findElement(By.name("q")).submit(); // Envia a
busca
System.out.println(driver.getTitle()); // Mostra o título da página
driver.quit(); // Fecha o navegador
```

---

## Por que o WebDriver é importante?

- Ele permite **testes automatizados de interface gráfica (UI)**, imitando o comportamento real de um usuário.
  - Você pode rodar os testes de forma automática, sem precisar testar tudo manualmente.
  - É essencial para **testes end-to-end (E2E)** em aplicações web modernas.
- 

## Como adicionar o JUnit ao seu projeto

**Se você estiver usando Maven, adicione ao `pom.xml`:**

Para JUnit 4:

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Para JUnit 5 (mais moderno):

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
```

⚠ Atenção: Se quiser usar JUnit 5, seu código precisará usar as novas anotações (@BeforeEach, @AfterEach, @Test, etc.).

---

## Se não estiver usando Maven:

1. Baixe o JUnit manualmente:
    - o [JUnit 4](#)
    - o [Hamcrest](#)
  2. Adicione os .jar ao classpath do projeto:
    - o Clique com o botão direito no projeto > **Build Path** > **Configure Build Path** > **Libraries** > **Add External JARs**
- 

## ✓ Exemplo com JUnit 4

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;

public class MeuTesteComSelenium {

    WebDriver driver;

    @Before
    public void setUp() {
        System.setProperty("webdriver.edge.driver",
"C:\\drivers\\msedgedriver.exe");
        driver = new EdgeDriver();
    }

    @Test
    public void testeTituloDoGoogle() {
        driver.get("https://www.google.com");
        String titulo = driver.getTitle();
        assertEquals("Google", titulo);
    }

    @After
    public void tearDown() {
```

```
        driver.quit();
    }
}
```

---

## Testar redirecionamentos com Selenium

### ✓ 1. Verificar a URL atual com `getCurrentUrl()`

Esse é o jeito mais direto:

```
driver.get("http://127.0.0.1:5500/login.html");

// faz algo que deve redirecionar
driver.findElement(By.id("entrar")).click();

// verifica se a URL mudou
assertEquals("http://127.0.0.1:5500/dashboard.html",
driver.getCurrentUrl());
```

---

### ✓ 2. Verificar o título da nova página com `getTitle()`

Se a nova página tiver um título único:

```
assertEquals("Página Principal", driver.getTitle());
```

---

## 3. Verificar a presença de um elemento exclusivo da nova página

```
// Espera até que um elemento da nova página esteja visível
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(5));
WebElement dashboardLabel = wait.until(
    ExpectedConditions.visibilityOfElementLocated(By.id("bemvindo"))
);

// Verifica se o texto está correto
assertEquals("Bem-vindo, Ana!", dashboardLabel.getText());
```

---

## Dica importante:

Se o redirecionamento for feito **com JavaScript assíncrono**, você **deve sempre usar `WebDriverWait`**, ou o teste pode falhar por "chegar antes da hora".

---

## Exemplo completo:

```
@Test
public void deveNavegarParaPaginaPrincipal() {
    driver.get("http://127.0.0.1:5500/login.html");
    driver.findElement(By.id("user")).sendKeys("Ana");
    driver.findElement(By.id("senha")).sendKeys("123");
    driver.findElement(By.tagName("button")).click();

    // Espera pela nova URL
    WebDriverWait wait = new WebDriverWait(driver,
    Duration.ofSeconds(5));

    wait.until(ExpectedConditions.urlToBe("http://127.0.0.1:5500/dashboard.html"));

    // Verifica se está na URL esperada
    assertEquals("http://127.0.0.1:5500/dashboard.html",
    driver.getCurrentUrl());
}
```

---

## Principais comandos do Selenium

### 1. Controle do Navegador

| Comando                                  | O que faz                                        |
|------------------------------------------|--------------------------------------------------|
| <code>driver.get("URL")</code>           | Acessa a URL especificada                        |
| <code>driver.navigate().to("URL")</code> | Navega para uma URL (igual ao <code>get</code> ) |
| <code>driver.navigate().back()</code>    | Volta para a página anterior                     |
| <code>driver.navigate().forward()</code> | Avança para a próxima página                     |
| <code>driver.navigate().refresh()</code> | Recarrega a página atual                         |
| <code>driver.quit()</code>               | Fecha o navegador e encerra o WebDriver          |
| <code>driver.close()</code>              | Fecha apenas a aba atual                         |

---

### ✓ 2. Informações da Página

| Comando                             | O que faz                  |
|-------------------------------------|----------------------------|
| <code>driver.getTitle()</code>      | Retorna o título da página |
| <code>driver.getCurrentUrl()</code> | Retorna a URL atual        |



| Comando                             | O que faz                      |
|-------------------------------------|--------------------------------|
| <code>driver.getPageSource()</code> | Retorna o HTML da página atual |

### ✓ 3. Localizar Elementos

| Comando                                                | O que faz                               |
|--------------------------------------------------------|-----------------------------------------|
| <code>driver.findElement(By.id("id"))</code>           | Localiza elemento pelo ID               |
| <code>driver.findElement(By.name("name"))</code>       | Pelo atributo <code>name</code>         |
| <code>driver.findElement(By.className("class"))</code> | Pela classe CSS                         |
| <code>driver.findElement(By.tagName("tag"))</code>     | Pela tag HTML                           |
| <code>driver.findElement(By.xpath("xpath"))</code>     | Usando XPath                            |
| <code>driver.findElement(By.cssSelector("css"))</code> | Usando seletor CSS                      |
| <code>driver.findElements(...)</code>                  | Retorna uma lista de elementos (plural) |

### ✓ 4. Ações em Elementos

| Comando                                | O que faz                           |
|----------------------------------------|-------------------------------------|
| <code>.click()</code>                  | Clica em um botão, link, etc.       |
| <code>.sendKeys("texto")</code>        | Digita texto em um campo            |
| <code>.clear()</code>                  | Limpa o campo de texto              |
| <code>.submit()</code>                 | Envia um formulário                 |
| <code>.getText()</code>                | Pega o texto visível do elemento    |
| <code>.getAttribute("atributo")</code> | Retorna valor de um atributo HTML   |
| <code>.isDisplayed()</code>            | Verifica se o elemento está visível |
| <code>.isEnabled()</code>              | Verifica se está habilitado         |

| Comando                    | O que faz                               |
|----------------------------|-----------------------------------------|
| <code>.isSelected()</code> | Verifica se está selecionado (checkbox) |

## ✓ 5. Esperas (Waits)

```
WebDriverWait wait = new WebDriverWait(driver,
Duration.ofSeconds(10));
```

| Comando                                                               | O que faz                            |
|-----------------------------------------------------------------------|--------------------------------------|
| <code>wait.until(ExpectedConditions.visibilityOf(...))</code>         | Espera até o elemento ficar visível  |
| <code>wait.until(ExpectedConditions.elementToBeClickable(...))</code> | Espera até o elemento estar clicável |
| <code>wait.until(ExpectedConditions.urlToBe("url"))</code>            | Espera até a URL ser a esperada      |

## ✓ 6. Comandos Extras

| Comando                                                     | O que faz                                   |
|-------------------------------------------------------------|---------------------------------------------|
| <code>driver.manage().window().maximize()</code>            | Maximiza a janela                           |
| <code>driver.switchTo().alert()</code>                      | Alterna para um alerta JavaScript           |
| <code>driver.switchTo().frame("idOuNome")</code>            | Entra em um <iframe>                        |
| <code>driver.switchTo().defaultContent()</code>             | Volta ao conteúdo principal (sai do iframe) |
| <code>driver.manage().timeouts().implicitlyWait(...)</code> | Define espera implícita padrão              |

## ✓ 7. Ações Avançadas (mouse, teclado)

Use a classe Actions:

```
Actions actions = new Actions(driver);
```

| Comando                                        | O que faz                   |
|------------------------------------------------|-----------------------------|
| <code>.moveToElement(elem).perform()</code>    | Move o mouse até o elemento |
| <code>.doubleClick(elem).perform()</code>      | Clique duplo                |
| <code>.contextClick(elem).perform()</code>     | Clique com o botão direito  |
| <code>.dragAndDrop(src, dest).perform()</code> | Arrasta de um para outro    |